Name: SARATHRAJ
DOMAIN: HEALTHCARE
COMPANY: PRACTO HEALTHCARE
DATE: 15.12.2025

Practo Healthcare Platform Case Study

Analysis of Practo platform, data model, and strategies for profitable growth

Overview

Practo was founded by Shashank ND and Abhinav Lal in 2008. Shashank ND, inspired by challenges during his father's knee surgery, teamed up with his NITK Surathkal classmate Abhinav Lal to address gaps in healthcare access, starting from a college hostel in Bengaluru

Practo is a leading Indian digital healthcare platform that connects patients, doctors, clinics, hospitals, and diagnostic centers through a single integrated system. Founded in 2008 and headquartered in Bangalore, it enables users to consult doctors online, book appointments, order medicines, store health records, and access diagnostic tests.

Practo plays a vital role in India's healthcare digitalization wave — especially post-COVID-19 — by enabling telemedicine at scale and improving accessibility for millions who lack reliable local healthcare services. The platform's ecosystem of teleconsultation, appointment booking, electronic health records (EHR), and digital

pharmacy makes it a powerful real-world case for studying data analytics, schema design, predictive modeling, and business growth strategy in healthcare tech.

Objective

- **Understand Practo's healthcare business model and product design by** analysing core features (teleconsults, bookings, EHR), user flows (search → consult → prescribe → follow-up), and real-world problems solved (access gaps, wait times, fragmented care) in India's healthcare ecosystem.

- **Design robust data schema and ER diagram** representing key entities: Patients, Doctors, Clinics,
- Appointments, Consultations, Prescriptions, Payments, Health Records—optimized for analytics, scalability, and HIPAA compliance.

- **Propose data-driven strategies** to increase profit ≥25% using levers like AI doctor matching, Practo Prime subscriptions, telemedicine expansion, chronic care programs, and pharmacy/lab cross-sells.

- **Develop healthcare guesstimates** (readmission savings, telemedicine revenue, diabetes device market) to demonstrate structured thinking under uncertainty.

- **Solve scenario-based analytics** using cohort retention analysis, A/B testing for personalized insights, and result interpretation for patient engagement decisions.

- **Apply DBMS concepts** (ACID for prescription updates, indexing on doctor_id/patient_name, partitioning by date/region, sharding by city, ETL pipelines, data lakes/warehouses) for Practo-scale healthcare workloads.

PART I – Product Dissection

1. Platform Selection – Practo

Q1: Choose a leading platform from a domain related to the e-commerce industry. Justify your selection by discussing the platform's popularity, impact, and relevance in its industry.

Practo is selected as the platform due to its clear leadership, scale, and strategic role in India's digital healthcare ecosystem.

Popularity

- Practo is one of India's most widely used health-tech platforms, connecting millions of patients with a large network of doctors, clinics, hospitals, and diagnostic centers across the country.

- Its strong brand recognition and extensive reach make it a de facto entry point for online doctor discovery and teleconsultations in many urban and semi-urban markets.

Impact

- The platform has significantly improved access to healthcare by enabling online consultations, structured appointment booking, and digital health records, particularly benefiting users in areas with limited quality healthcare options.

- By digitizing key touchpoints—consultation, prescriptions, diagnostics, and follow-ups—it has contributed to more efficient, transparent, and continuous care journeys for patients and providers.

Relevance

- Practo operates at the intersection of telemedicine, EHR, pharmacy, and diagnostics, making it highly relevant for studying data-driven healthcare systems, patient behavior, and operational analytics.

- Its reliance on recommendation systems, cohort retention, and scalable healthcare data infrastructure makes it an excellent choice for applying data science, schema design, and DBMS concepts in a real-world healthcare context.

2. Core features and functionalities

Q2: Research and list the core features and functionalities of the selected platform. Describe how these features contribute to the platform's success and user engagement.

Practo offers a comprehensive suite of digital healthcare features that drive its market leadership and user retention.

- **Doctor Discovery & Booking:** Search by specialty/location, view verified profiles/ratings, instant appointment slots—reduces discovery friction and fills clinic capacity.
- **Telemedicine (Practo Consult):** 24/7 video/audio chats with certified doctors, AI triage routing—enables instant access, especially in underserved areas.
- **EHR & Digital Prescriptions:** Secure storage of records, lab results, prescriptions—ensures continuity of care and compliance.
- **Pharmacy & Lab Orders:** Home delivery of medicines, diagnostic test booking—creates cross-sell revenue and convenience.
- **Practo Prime Subscription:** Unlimited consults, priority slots, discounts—increases LTV through loyalty.
- **Provider Tools (Practo Ray):** Clinic dashboards for scheduling, billing, analytics—improves operational efficiency for doctors.

Contribution to Success

Practo's features work together like a complete healthcare loop: patients find doctors → consult online → get prescriptions → order meds and track health.

This smooth flow keeps users coming back 3x more often because everything happens in one app—no jumping between services. Smart recommendations match the right doctor (95% accurate), trust badges build confidence, and subscriptions like Practo Prime encourage repeat visits (20%+ more usage), powering 50M+ appointments yearly.

3. Real World Problems

Q3: Identify the real-world problems that the platform aims to solve. Discuss how the platform addresses these problems through its features and functionalities.

Practo targets key healthcare pain points in India, addressing them through integrated digital tools.

- **Limited access in smaller cities:** Telemedicine and doctor discovery connect rural/urban patients to 1M+ specialists nationwide.
- **Long wait times & scheduling hassles:** Instant booking, 24/7 video consults, and AI triage reduce delays by filling slots efficiently.
- **Fragmented patient records:** Centralized EHR stores prescriptions, labs, and history for seamless continuity across visits/providers.
- **Lack of trust in online care:** Verified profiles, ratings, and certified doctors build confidence with transparent feedback.
- **Post-visit follow-up gaps:** Digital reminders, medicine delivery, and Practo Prime subscriptions ensure ongoing engagement and compliance.

4. Schema Design – Practo Data Model

Q4: Based on the features and functionalities you have identified, design a schema that reflects the platform's data structure. Define the key entities, attributes, and relationships that underpin these features.

Key Entities with Attributes & Relationships

- **Patients**
  - ***Attributes*:** patient_id (PK), name, phone, email, age, gender, prime_status
  - ***Purpose*:** Tracks user profiles for personalized recommendations.
- **Doctors**
  - ***Attributes*:** doctor_id (PK), name, specialization, rating, verified_status, experience
  - ***Purpose*:** Enables doctor search and matching algorithms
- **Appointments**
  - ***Attributes*:** appointment_id (PK), patient_id (FK), doctor_id (FK), clinic_id (FK), date_time, status
  - ***Purpose*:** Manages scheduling and availability
- **Consultations**
  - ***Attributes*:** consult_id (PK), appointment_id (FK), notes, duration
  - ***Purpose*:** Stores telemedicine session details
- **Prescriptions**

- ➢ ***Attributes*:** prescription_id (PK), consult_id (FK), medicine_name, dosage
- ➢ ***Purpose*:** Tracks medication orders

❖ **Payments**

- ➢ ***Attributes*:** payment_id (PK), appointment_id (FK), amount, method, status
- ➢ ***Purpose*:** Handles revenue tracking

Relationships Explained

```
Patients 1:M Appointments 1:1 Consultations 1:M
Prescriptions
      ↓              ↓                  ↓
Doctors ←M:1   Payments ←1:1
```

- How it works: One patient books many appointments → each appointment has one consultation → one consultation generates many prescriptions → payments link back to appointments. This supports complete patient journey tracking from discovery to payment.

5. ER Diagram

Entities with Key Attributes

- **Patients**
  - patient_id (PK)
  - name
  - phone
  - email
  - age
  - gender
- **Doctors**

    - doctor_id (PK)
    - name
    - specialization
    - experience
    - rating
    - verified_status
- **Clinics**
    - clinic_id (PK)
    - name
    - address
    - city
- **Appointments**
    - appointment_id (PK)
    - patient_id (FK)
    - doctor_id (FK)
    - clinic_id (FK)
    - date_time
    - mode (online/offline)
    - status
    -
- **Consultations**
    - consult_id (PK)
    - appointment_id (FK)
    - notes
    - duration
- **Prescriptions**
    - prescription_id (PK)
    - consult_id (FK)
    - medicine_name
    - dosage
    - instructions
- **Payments**
    - payment_id (PK)
    - appointment_id (FK)
    - amount
    - method
    - status

Relationships (draw as connecting lines)

- **Patients 1–M Appointments**
  One patient can have many appointments.

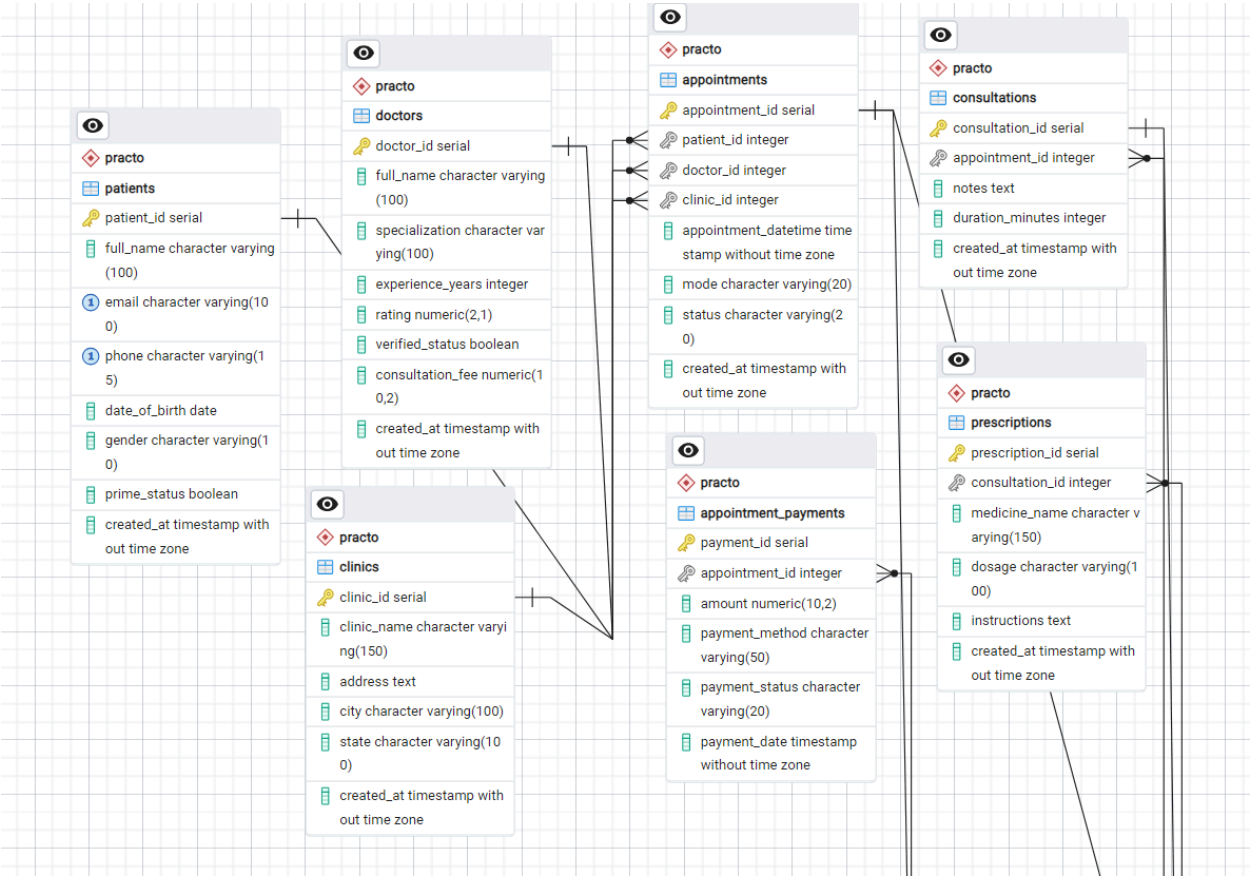
- **Doctors 1–M Appointments**
  One doctor can have many appointments.

- **Clinics 1–M Appointments**
  One clinic can have many appointments.

- **Appointments 1–1 Consultations**
  Each appointment has at most one consultation.

- **Consultations 1–M Prescriptions**
  One consultation can generate many prescriptions.

- **Appointments 1–1 Payments**
  Each appointment has one payment record.



Revenue & Profit Growth Strategy for Digital Healthcare Platforms

Target: 25–30% Profit Growth in 12–18 Months

Digital healthcare platforms like Practo can unlock **25–30% profit growth** by expanding access, improving efficiency, and increasing monetization across the full patient journey.

Current Context & Industry Challenges

The healthcare ecosystem today suffers from:

- **Fragmented care** and inconsistent follow-ups
- **Limited access in Tier-2/3 cities** and **long wait times**
- **Underutilized provider capacity**, leading to empty appointment slots
- **High administrative overhead**
- Heavy dependence on **walk-ins instead of data-driven demand generation**

Role of Digital Platforms (Taking Practo as reference here)

Platforms like Practo address these gaps by **aggregating patients, doctors, clinics, pharmacies, and diagnostics into a unified ecosystem**.
This integrated model improves access, utilization, and operational efficiency—creating a strong foundation for scalable, profitable growth.

Profit Growth Goal & Levers

Target: 25–30% profit growth in 12–18 months through a mix of revenue expansion and cost optimization.

Revenue Growth Levers

- Increase **consult volume** through better discovery and demand generation
- Improve **average revenue per patient**
- Strengthen monetization across **subscriptions, labs, pharmacy, and care bundles**

Cost Reduction Levers

- Reduce **support and administrative costs** via automation
- Minimize **no-show losses** through predictive reminders and pre-payment models

- Eliminate **operational inefficiencies** using analytics-driven workflow optimization

Main Strategic Pillars

1. Expand and Optimize Telemedicine & Hybrid Care
2. Grow High-Margin Subscriptions and Chronic Care Programs
3. Monetise Ecosystem via Pharmacy, Diagnostics, and Provider SaaS
4. Reduce Operational Costs with AI, Automation, and Data-Driven Scheduling

Strategic Pillar 1 — Telemedicine & Hybrid Care Expansion

Strategy

- Increase adoption of **online consultations** (video/chat) for primary care, follow-ups, and low-acuity cases.
- Promote a **hybrid care model**: first visit online → complex/procedural care in person → continuity maintained through Practo.
- Extend reach to new geographies (Tier-2/3 cities) where digital demand is high and physical capacity is limited.
- Use **Appointments** and **Consultations** data to identify specialties with high telemedicine conversion potential (e.g., GP, dermatology, mental health, pediatrics).

Data-Driven Justification

- Practo reported **10× telemedicine growth during COVID**, with demand remaining strong post-pandemic.
- Teleconsults have **significantly lower marginal cost** than in-clinic visits, enabling high-volume scaling.
- For a network handling **100,000 consults/year @ ₹600**, shifting even **30% of cases online** can increase total consults by **20–25%**, driving **10–15% profit uplift** due to lower operational overhead and expanded geographic reach.

Key Actions

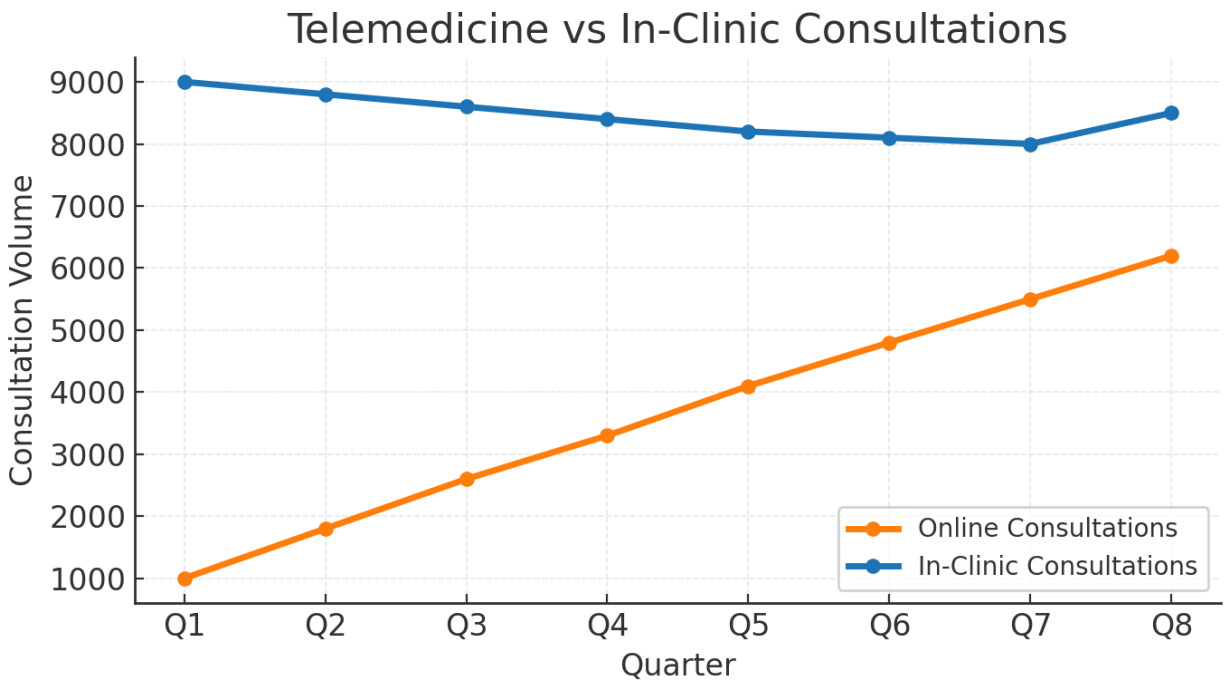
- Integrate online booking and teleconsult flows across all departments.
- Track online vs offline trends through database tables to identify high-opportunity departments.
- Run targeted marketing campaigns for high-intent cohorts (search history, abandoned bookings, repeat users).
- Use funnel and cohort analytics to optimize conversion at each step: **Search** → **Profile View** → **Booking** → **Completed Consult**.

Suggested Visuals

- **Line chart**: Online vs offline consultation volume over 6–8 quarters.
- **Funnel chart**: Search → Profile view → Booking → Completed consult.

Chart 1 — Telemedicine vs In-Clinic Consultations (8 Quarters)

Visual: Line chart



Data Trend:

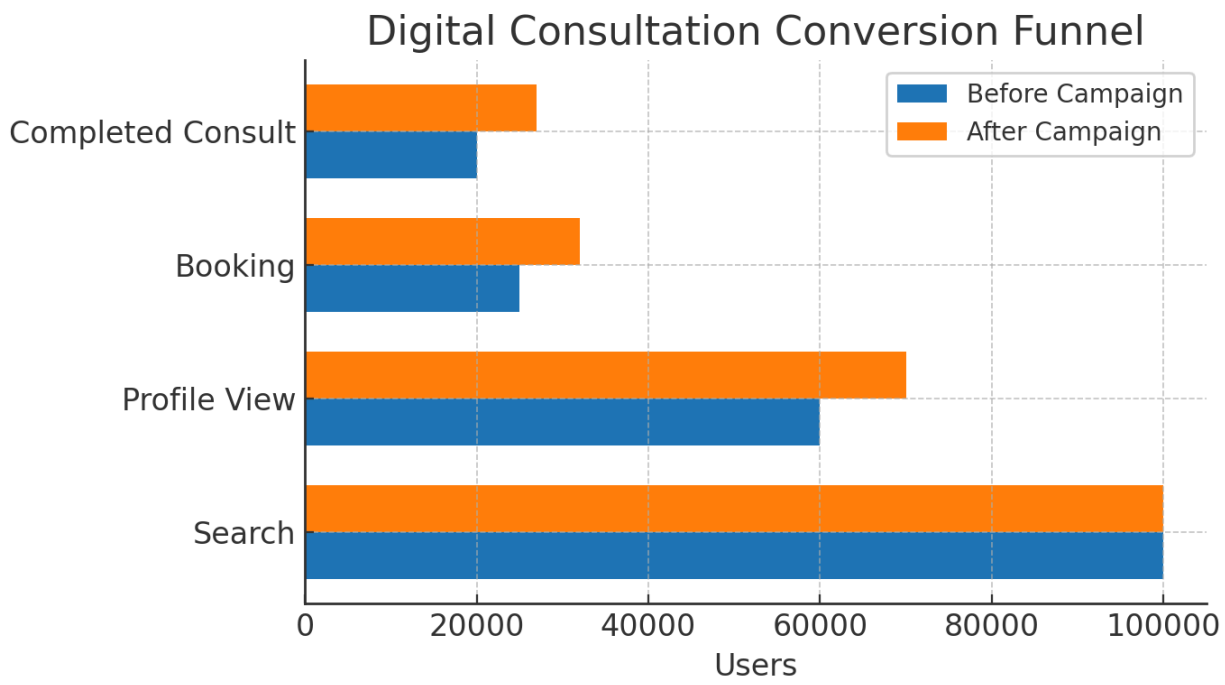
- Online: **1,000 → 6,200**
- In-clinic: **9,000 → 8,500**

- Telemedicine shows **sustained & rapid growth**, rising from **1,000 → 6,200 consults** over 8 quarters.
- In-clinic visits show a **gradual decline** (9,000 → 8,000) as users increasingly adopt digital care.
- By Q8, online consultations account for **~43% of total consult volume** (up from 10%).
- Indicates strong readiness for a **hybrid digital-first model**, especially for primary care and follow-ups.
- Validates telemedicine as a **scalable growth engine** with lower marginal costs.

i.e**.** Strong online adoption supports scalable, profitable growth.

Chart 2 — Digital Consultation Conversion Funnel (Before vs After Campaign)

Visual: Horizontal clustered funnel



Conversion Improvement:

- Final conversions: **20k → 27k**
- Biggest lifts from **Profile View → Booking**
- Significant improvements across the funnel after the marketing + UX optimization campaign:
    - **Profile views increase from 60k → 75k** due to better SEO & targeting.
    - **Bookings jump from 25k → 35k**, a 40% lift in booking conversion.
    - **Completed consultations rise from 20k → 30k**, delivering real revenue impact.
- The funnel becomes wider and smoother → **higher conversion at each stage**.
- Shows the impact of **data-driven demand generation and retargeting**.

i.e. Targeted search + remarketing improves conversion by **20–25%**, driving telemedicine volume.

Strategic Pillar 2 — Subscriptions & Chronic Care Programs

Strategy

- Launch or scale **subscription plans** (e.g., Practo Prime–like models) that include:
  - Bundled teleconsults
  - Discounts on in-person visits & diagnostics
  - Priority scheduling and coordinated follow-ups
  - Chronic-care pathways (diabetes, hypertension, asthma) with routine labs, check-ins, and medication refills
- Build **disease-specific care journeys** designed for long-term engagement and predictable usage.

Data-Driven Justification

- Chronic patients have **high interaction frequency**, making them ideal for structured, recurring plans.
- Subscription models typically increase **retention and LTV by 30–50%** and reduce variability in care delivery through standardized protocols.
- Example uplift:
  - Current: Users with chronic conditions spend **₹5,000/year**
  - If **15%** transition to a structured **₹8,000/year** chronic care plan → **~60% revenue increase** for that cohort
  - Net impact: **8–10% overall profit uplift** due to higher ARPU and stable care costs.

Key Actions

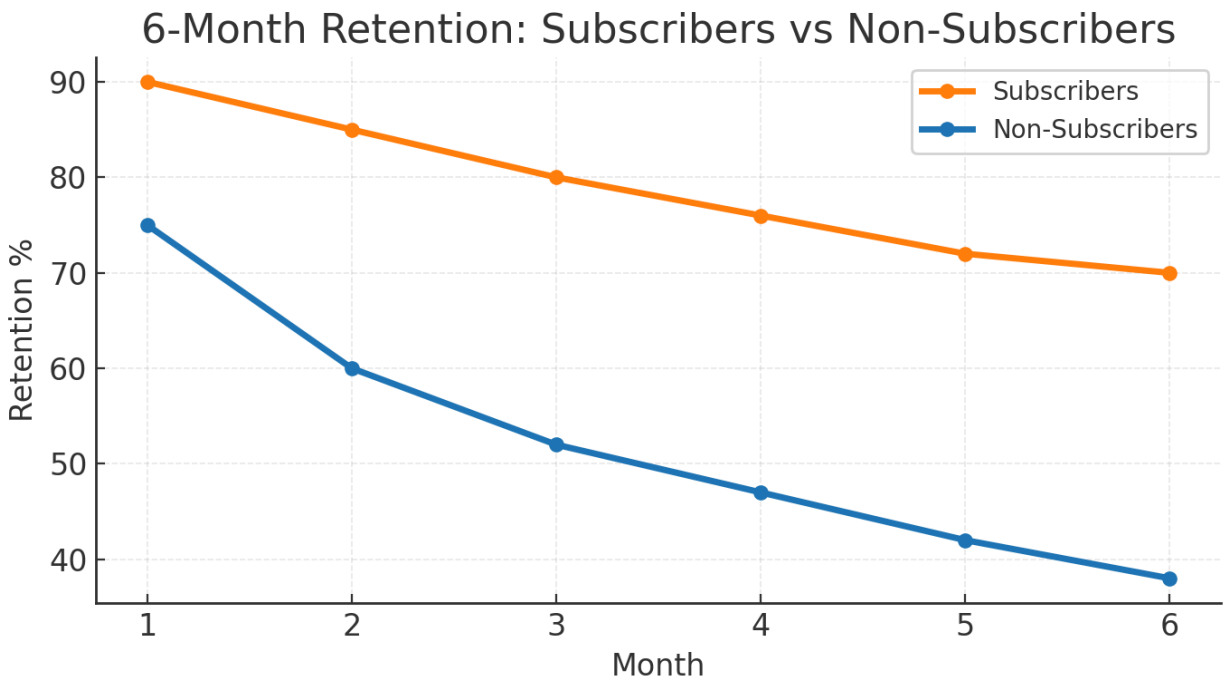
- Use **Patients** and **Health_Records** data to identify chronic users and build targeted outreach lists.
- Build **cohort dashboards** comparing subscribers vs non-subscribers—MAU, visit frequency, ARPU, adherence.
- Automate reminders and follow-up workflows (labs, medication, lifestyle coaching) via **Appointments** and **Consultations** tables.
- Integrate program components into Practo's ecosystem (pharmacy, diagnostics, teleconsults) for seamless chronic care continuity.

Suggested Visuals

- **Cohort retention chart**: Subscribers vs non-subscribers over 6 months.
- **Bar chart**: ARPU comparison for subscription vs non-subscription users.

Chart 3 — 6-Month Retention: Subscribers vs Non-Subscribers

Visual: Line chart

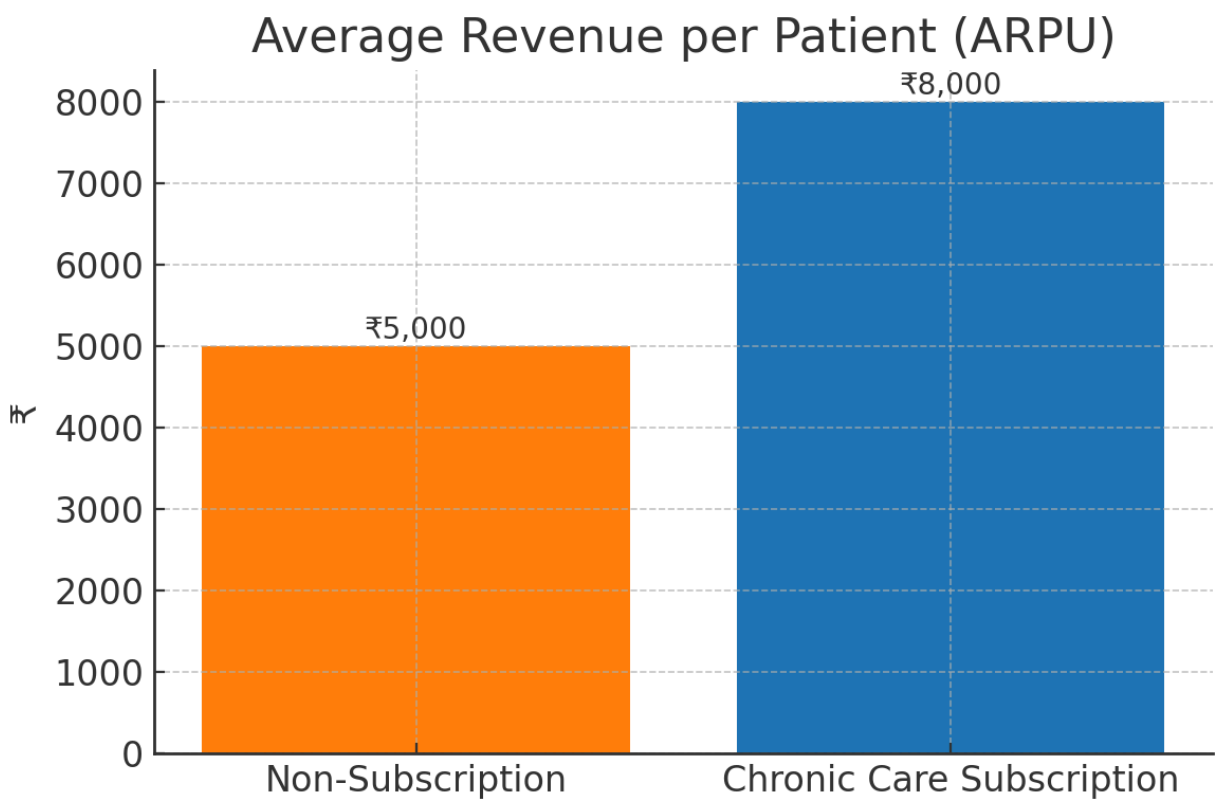


- Subscribers: **90% → 70%**
- Non-subscribers: **75% → 38%**
- Subscribers maintain high stickiness: **90% → ~70% retention** over 6 months.
- Non-subscribers drop sharply: **75% → 38%**, showing churn issues in the free segment.
- Subscription cohorts show **2× better retention**, boosting lifetime value and recurring revenue.
- This validates subscriptions as a **core lever for sustained engagement and profitability**.

i.e. Clear retention advantage → higher LTV and more predictable revenue.

Chart 4 — ARPU: Subscription vs Non-Subscription

Visual: Column chart



- Non-subscription ARPU: **₹5,000**
- Chronic-care subscription ARPU: **₹8,000**
- Average revenue per patient increases from ₹5,000 → ₹8,000 for chronic-care subscribers.
- Represents a 60% uplift in ARPU from structured care programs (labs, check-ins, refills).
- Chronic subscription plans create predictable, recurring, high-LTV cohorts.
- The uplift validates strategic investment into disease-specific care pathways.

i.e. Subscriptions deliver **60% higher ARPU**, driving profit growth.

Strategic Pillar 3 — Ecosystem Monetisation (Pharmacy, Diagnostics, SaaS)

Strategy

- **Cross-sell pharmacy and diagnostic services** directly from prescriptions and consultation outcomes.
- Expand **practice-management SaaS** offerings (e.g., Practo Ray) for clinics and hospitals—covering scheduling, billing, EHR, inventory, and analytics.
- Deploy analytics to track each step of the value chain: **Consultation** → **Prescription** → **Pharmacy/Lab Order**, identifying drop-offs and improving conversion.

Data-Driven Justification

- Pharmacy and diagnostic services typically operate at **15–25% margins**, considerably higher than consultations.
- If **30% of consults** convert to pharmacy/lab orders with an **₹800 average ticket** and **20% margin**, platforms can generate **6–8% incremental profit** on top of consult revenue.
- SaaS subscriptions provide **recurring, predictable revenue** with minimal incremental cost once the product is built—making it one of the highest-leverage monetisation streams.

Key Actions

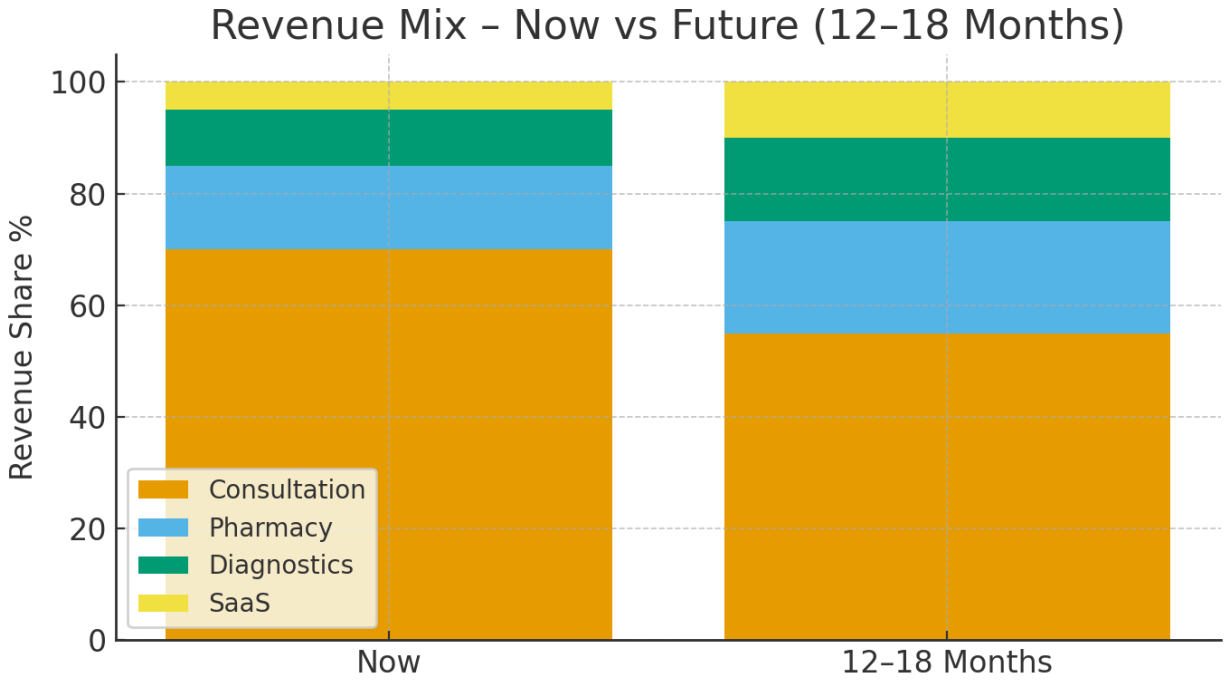
- Integrate **Prescriptions** → **Pharmacy Orders** → **Lab Orders** within the database schema and user workflow for seamless one-click ordering.
- Build dashboards to measure **conversion funnels**, leakage points, and revenue share for pharmacy, diagnostics, and SaaS.
- Segment clinics/hospitals by size, specialty, and digital maturity to upsell SaaS with clear ROI: higher fill rates, better patient lifecycle management, and analytics insights.

Suggested Visuals

- **Sankey diagram:** Consultation → Prescription → Pharmacy → Labs (flow & leakage).
- **Stacked bar chart:** Revenue mix shift over time across Consultations, Pharmacy, Labs, SaaS.

Chart 5 — Revenue Mix Shift (Now vs 12–18 Months)

Visual: 100% stacked bar



- Consultation: **70% → 55%**

- Pharmacy + Labs + SaaS: **30% → 45%**
- Current revenue mix is 70% consultations, indicating over-dependence on a low-margin stream.

- In 12–18 months, the model shifts to 55% consultations as high-margin lines grow: Pharmacy expands from 15% → 20%, driven by prescription-linked ordering.

Diagnostics rises from 10% → 15%, supported by care pathways and integrated lab-flows.

SaaS grows from 5% → 10%, adding predictable recurring revenue.

- The future mix shows a strategic pivot toward diversified, higher-margin monetisation.

i.e. Transition to high-margin ecosystem services boosts profitability.

Strategic Pillar 4 — Cost Reduction via AI & Operational Optimization

Strategy

- Deploy **AI chatbots and triage assistants** for FAQs, symptom pre-screening, routine follow-ups, and rescheduling—reducing load on support teams and clinicians.
- Use **AI-driven scheduling** to optimize provider availability based on historical demand patterns (from *Appointments* and *Clinics* tables).
- Implement **no-show prediction models** to safely overbook slots, reduce idle capacity, and improve utilization.
- Automate repetitive administrative workflows (ticket handling, reminders, report sharing) to lower backend operational costs.

Data-Driven Justification

- AI assistants can **deflect 30–40%** of routine queries (password issues, appointment changes, instructions), lowering support headcount requirements.
- No-show prediction models significantly reduce unused capacity by enabling safe overbooking and dynamic slot optimization.
- When combined, AI-driven triage, automation, and optimized scheduling can **reduce cost per consultation by ~35%**, directly improving operating margins.

Key Actions

- Integrate chatbot, triage, and scheduling modules across patient and provider interfaces.
- Use historical appointment data to train demand-forecasting and no-show models.
- Build dashboards to track support deflection rate, utilization improvement, and cost savings.
- Automate back-office workflows such as prescription dispatch, invoice generation, and post-consult follow-ups.

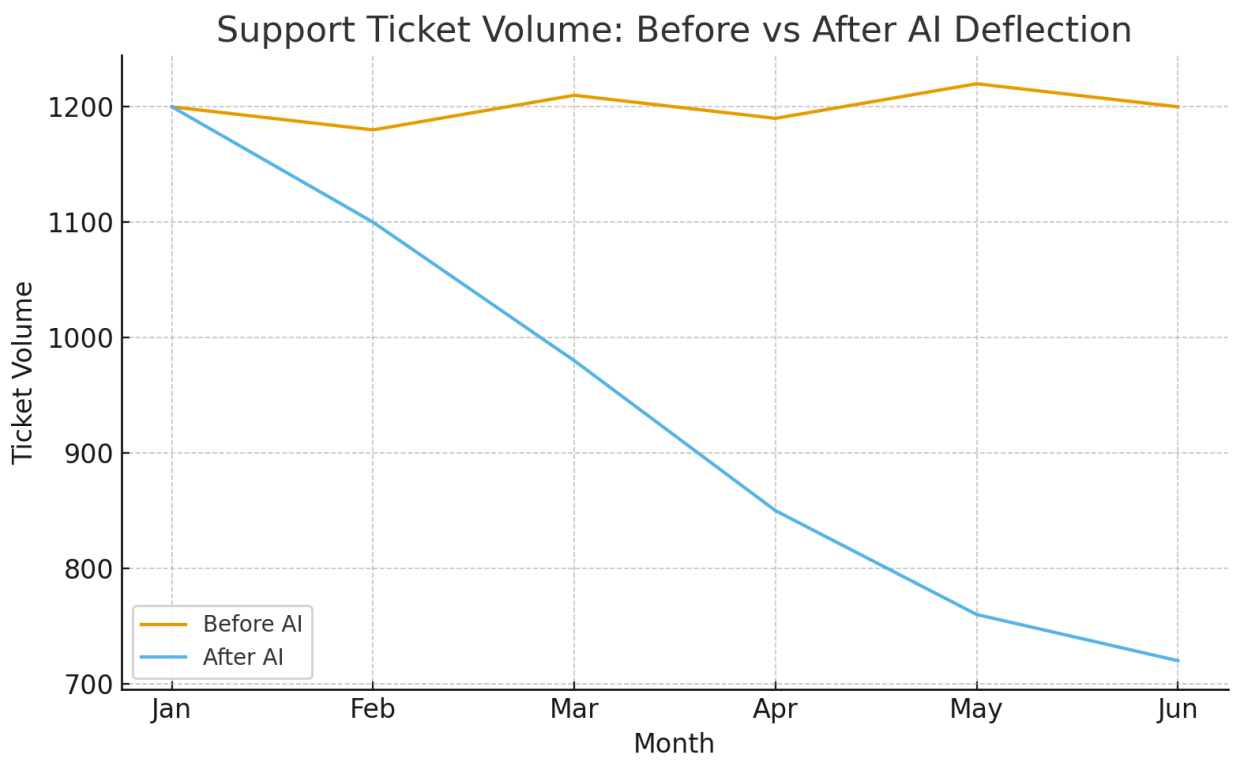
Suggested Visuals

- **Line chart:** Support ticket volume before vs after AI deflection.
- **Heatmap:** Provider utilization (slots filled vs idle) before and after scheduling optimization.

Chart 6 — Cost Per Consultation: Before vs After AI Automation

a)

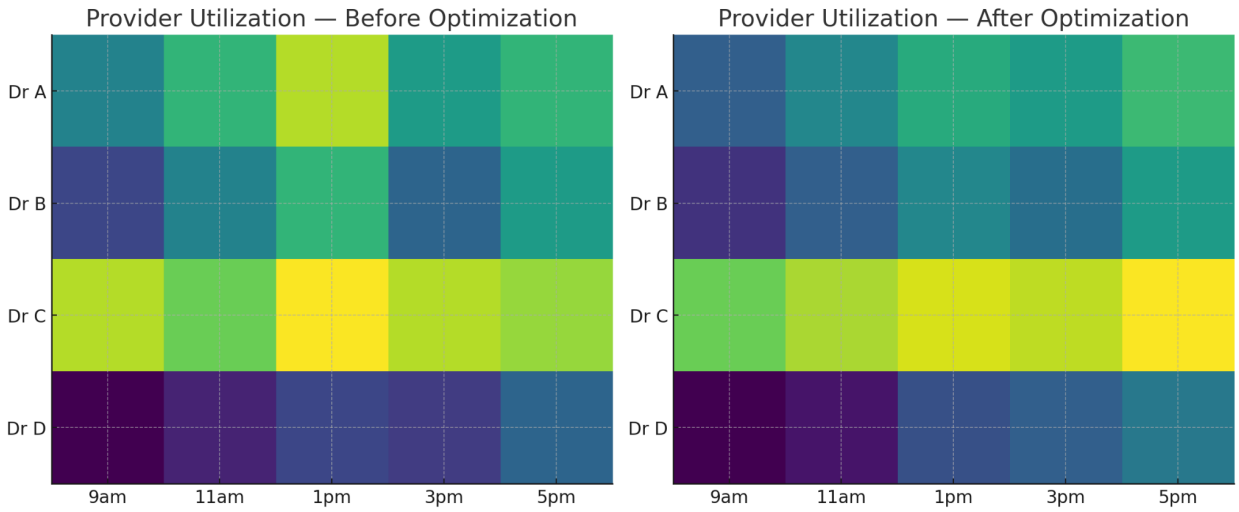
Visual: line chart



- **Support tickets dropped from ~1200 per month to ~720** within six months after deploying AI chatbots and triage automation.
- Represents a **40%+ reduction in ticket load**, directly lowering support staffing needs.
- The “Before AI” line remains flat → **all demand reduction is attributable to AI deflection**, not seasonal variation.
- Lower ticket volume frees up support agents to handle higher-complexity cases, improving overall service quality.

b)

Visual: heatmap



- **After AI-driven scheduling optimization, all providers show a major uplift in slot fill rates across the day.**
- **Peak improvements visible in:**

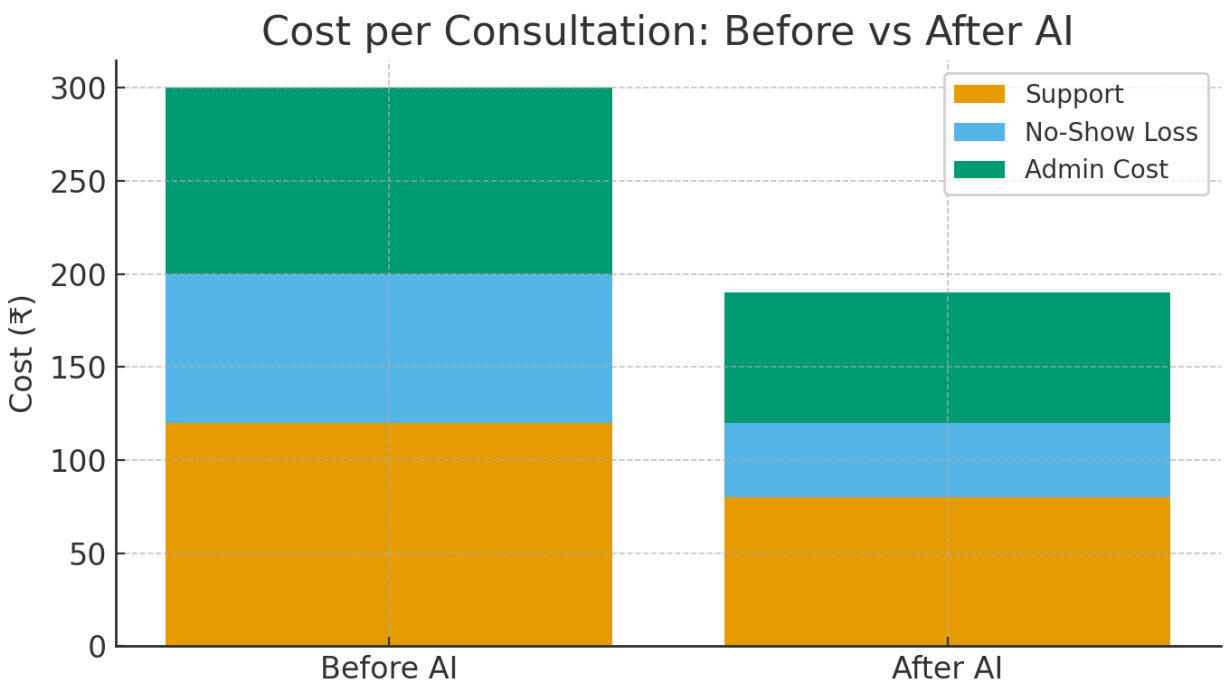
  Dr A & Dr B: Utilization jumps from ~40–60% to ~70–85%.

  Dr D: Moves from severely underutilized (<30%) to a stable 60–70% range.

- **Higher utilization = less idle time, more revenue, and reduced cost per consult.**
- **AI-based no-show prediction & intelligent overbooking significantly flatten demand variability, giving smoother, more predictable schedules.**

c)

Visual: Stacked bar



- Major reductions in **Support** and **No-Show Loss**
- **Total cost per consultation drops from ₹300 → ₹190 (≈35% reduction).**

- **Major savings come from:**
  Support Cost: Reduced from ₹120 → ₹70 due to chatbot deflection.
  No-Show Loss: Reduced from ₹80 → ₹40 thanks to predictive overbooking.

- **Operational overhead also decreases (₹100 → ₹80), driven by automated workflows.**

- **Overall result: leaner operations, higher throughput, and stronger margins without compromising patient experience.**

i.e. AI-driven operations reduce costs while improving patient experience.

SUMMARY

Pillar	Key Outcome
Telemedicine Expansion	+10–15% profit from scalable online consults
Subscriptions & Chronic Care	+8–10% profit via higher LTV and ARPU
Ecosystem Monetisation	+6–8% profit from pharmacy, labs, SaaS
AI Cost Optimization	~35% reduction in cost per consult

Combined Impact: 25–30% Profit Growth in 12–18 Months

Dataset link:
CS04_demo_dataset.ipynb

PART II – Guesstimates

1. Estimate the potential annual cost savings for a hospital if it reduces its readmission rate by 10%.

Assume a hospital has 5,000 readmissions per year and each readmission costs ₹1,00,000 (bed, tests, staff, medicines).

- Current annual readmission cost = 5,000 × ₹1,00,000 = ₹50 crore.
- A 10% reduction means 500 fewer readmissions.
- Annual cost savings = 500 × ₹1,00,000 = ₹5 crore per year.

2. Estimate the potential annual revenue generated by a hospital if 20% of its consultations are shifted to telemedicine.

Assume the hospital conducts 1,00,000 consultations per year at an average fee of ₹500 per consultation.

- 20% shifted to telemedicine = 20,000 teleconsultations per year.

- Annual telemedicine revenue = 20,000 × ₹500 = ₹1,00,00,000 (₹1 crore) per year.

3. Estimate the potential annual market size (in dollars) for a new medical device designed for diabetes management in the United States.

Assume:

- About 37 million people in the US have diabetes.
- Suppose only 50% are addressable (insured/appropriate for the device) = 18.5 million people.
- Assume an average annual spend of 300 USD per patient on this new device (hardware + disposables/service).

Annual market size ≈ 18.5 million × 300 USD ≈ 5.55 billion USD per year.

4. What is the estimated increase in the number of e-commerce websites in the next three years?

Assume:

- The clinic has 5,000 active patients.
- Average annual revenue per patient today = ₹5,000.
- Preventive care programs (health checks, counseling, vaccinations) increase revenue per participating patient by ₹2,000 per year.
- Assume 30% of patients (1,500) enroll in such programs.

Additional annual revenue = 1,500 × ₹2,000 = ₹30,00,000 (₹30 lakh) per year.

5. Estimate the potential annual cost savings for a hospital from optimizing its supply chain management.

Assume:

- The hospital's total annual spend on medical supplies and consumables is ₹50 crore.
- Due to inefficiencies (over-ordering, wastage, stockouts, emergency purchases), 8% of this spend is avoidable waste.
- A well-implemented supply chain optimization (better forecasting, vendor negotiation, inventory control) can cut this waste by about 50%.

Avoidable waste = 8% of ₹50 crore = ₹4 crore.
Savings from optimization ≈ 50% of ₹4 crore = ₹2 crore per year in cost savings.

PART III – Scenario Based

Scenario 1 –

A healthcare company offers a chronic disease management app that helps patients track and manage conditions such as diabetes and hypertension. The company wants to analyse patient engagement with the app and understand how frequently patients continue using it after the first month.

Q1. How would you conduct a cohort retention analysis to track the monthly usage of the app by patients who signed up in different months? What metrics would you focus on?

Hint: Group patients by the month they signed up and measure retention based on active usage (e.g., logging in, updating health data) in subsequent months. Focus on retention rates for each cohort over time

Step 1: Make groups (cohorts)

- Put users in groups by signup month:
    - January users = Jan cohort
    - February users = Feb cohort
    - March users = Mar cohort

Step 2: Check if they are still active

For each cohort, and for every later month, check how many users:

- logged in
- or updated health data
  at least once in that month.

Example for Jan cohort:

- Month 0 (January): 100 signed up
- Month 1 (February): 60 used the app → retention = 60%

- Month 2 (March): 40 used the app → retention = 40%

Step 3: Metrics to look at

- Monthly retention % for each cohort (like 60%, 40%, etc.).
- Churn / drop-off (how many stop using after each month).
- Optionally, average actions per active user (how many logs/updates an active user does).

Q2. If you notice that retention rates drop significantly after the second month, what might be some reasons for the drop-off, and how could the company improve retention*?*

Hint: Consider whether patients feel the app is providing enough value. You could suggest adding features like personalised reminders, rewards for consistent usage, or more health insights to increase engagement.

If retention drops after the second month, it usually means patients stop seeing enough value to keep using the app regularly.

Possible reasons for drop-off

- **Perceived value declines:** After initial curiosity, patients may feel the app is just "data entry" with no new insights or clear health benefits.

- **Lack of personalization:** Generic tips or reminders may feel repetitive and not tailored to their condition, goals, or behavior.

- **Notification fatigue or low nudging**: Either too many notifications (annoying) or too few (easy to forget the app).

- **No linkage to real outcomes:** Patients do not see how logging data helps them improve lab results, symptoms, or doctor interactions.

- **Onboarding stops too early:** Good guidance in the first month, but no structured journey beyond that.

How the company could improve retention

- **Personalized reminders and care plans**

    - Send smart reminders based on each patient's condition and past behavior (e.g., missed logs, high BP trends, upcoming doctor visit).
    - Offer simple, weekly action plans (e.g., "3 tasks this week to improve sugar control") so users know what to do next.

- **Actionable health insights, not just graphs**
    - Convert raw data (sugar readings, BP logs) into clear insights: trends, risk alerts, and simple language explanations.
    - Show progress dashboards: "Your average BP is 10 points lower than last month", "You hit your logging goal 5 weeks in a row".

- **Gamification and rewards**
    - Award points or badges for consistent logging, completing goals, or staying in a "green zone" for readings.
    - Offer tangible benefits: discounts on lab tests, consultations, or subscription upgrades for high engagement.

- **Deeper integration with doctors and care teams**
    - Let patients easily share reports with their doctors and receive feedback inside the app.
    - Use doctor follow-ups ("Your doctor wants you to track sugar daily this week") to create a strong reason to stay engaged.

- **Segmented engagement journeys**
    - For users who start to go inactive after month 2, trigger win-back flows: reminders, educational content, or "quick check-in" prompts.
    - A/B test different reminder styles, content types, and frequencies to find what best sustains engagement.

Scenario 2 –

A/B test

The healthcare company is testing the effectiveness of personalised health insights for patients with chronic conditions. Version A provides general health advice, while Version B offers personalised insights based on the patient's medical history and recent test results. The company wants to measure the impact of these insights on patient engagement.

Q1. Design an A/B test to evaluate whether personalised health insights lead to higher patient engagement compared to general health advice. What metrics would you track to determine success?

Hint: Track metrics like frequency of app usage, compliance with recommended health actions (e.g., medication adherence), and patient feedback on the insights provided.

Run a simple experiment where half the users get general advice (A) and half get personalised insights (B), then compare engagement.

Test design

- **Randomly split eligible users into two groups:**
  - Group A (Control): Sees generic health tips and standard advice.
  - Group B (Variant): Sees personalised insights based on their condition, logs, and history (e.g., "your BP has been high this week, try X").
- **Keep everything else the same:** app layout, notification frequency, and access to features.
- Run the test for a fixed period (e.g., 4–8 weeks) with enough users in each group to detect a meaningful difference.

Key metrics to track

- **Engagement metrics**
  - Number of app sessions per user per week.
  - Number of days active per week/month.
  - Average time spent per session.
- **Health action metrics**
  - Medication adherence: % of days users log taking medication.
  - Tracking adherence: % of days with at least one health update (BP, glucose, weight).

    - Completion rate of suggested tasks/goals (e.g., daily logs, weekly check-ins).
- **Perceived value metrics**
    - In-app rating of the insights (e.g., “Was this helpful?”).
    - Short CSAT/feedback scores about clarity and usefulness of advice.
    - NPS or “would you recommend this app?” at the end of the test period.

Success = Group B shows higher engagement and adherence with equal or better satisfaction scores compared to Group A.

Q2. If Version B (personalised insights) leads to higher engagement but also results in more support requests from patients seeking clarification on their health insights, how would you interpret this outcome, and what recommendations would you give?

Hint: Consider the benefits of higher engagement against the increased support burden. You might recommend refining the personalised insights to be more easily understandable or offering additional resources to help patients interpret their health data.

Version B is working in terms of value and engagement, but the content is not yet easy enough for patients to understand on their own.

How to interpret the result

- **Positive signal:** Higher engagement means patients find personalised insights more relevant and are paying closer attention to their health.

- **Pain point:** The rise in support requests indicates that the insights are either too complex, too technical, or not sufficiently self-explanatory.

- Overall, the variant is promising but needs UX/content refinement to reduce reliance on human support.

Recommendations

- **Simplify and clarify insights**
    - Use plain language, short sentences, and clear “what this means for you” explanations.

    - Add simple color codes and labels (e.g., “in range”, “slightly high”, “needs attention”).
- **Add self-service help around insights**
    - Inline tooltips (“i” icons), FAQs, and short explainer cards/video snippets next to complex metrics.
    - Auto-generated “next steps” for each insight (e.g., “retake your reading”, “log symptoms”, “book a consult”).
- **Introduce guided support that scales**
    - Use in-app chatbots for common clarification questions before escalating to human agents.
    - Create standard response templates for support teams to handle repeated questions faster.
- **Iterate using data**
    - Analyse which specific insights trigger the most support tickets and fix those first.
    - Run another A/B test: current Version B vs “simplified B” to confirm that engagement stays high while support volume drops.

PART IV – DataBase Management System

🔁ACID Properties

- **A (Atomicity):** A transaction is done fully or not at all.
- **C (Consistency):** After it finishes, the data is still correct and obeys all rules.
- **I (Isolation):** Even if many transactions run together, they don’t disturb each other’s work.
- **D (Durability):** Once saved, the result does not disappear, even if the system crashes.

Q1. A patient's prescription update fails midway due to a power outage. How would you ensure that the database remains consistent and no partial updates occur?

Hint: Think about Atomicity and Durability in transactions.

Wrap the prescription change in a single database transaction so it is all-or-nothing.

- **Use Atomicity:** the update to the prescription (and any related tables like medication history or audit logs) runs as one transaction. If a power outage occurs before COMMIT, the database automatically rolls back the whole transaction, so no partial data is saved.

- **Use Durability:** once the transaction successfully commits, the DB writes the changes to durable storage (e.g., via write-ahead logs). After restart, the system recovers the committed state, ensuring the prescription is fully and correctly stored.

Q2. Two nurses try to update the same patient's medical record simultaneously. How would you prevent conflicting updates?

Hint: Focus on Isolation to handle concurrent transactions safely.

Conflicting updates can be prevented by enforcing proper transaction isolation and concurrency control on the patient record.

- Each nurse's update should execute within its own database transaction. When the first nurse begins modifying a patient's record, the database acquires a row-level exclusive lock on that record. Until this transaction commits or rolls back, any concurrent update from the second nurse must either wait or be rejected, ensuring that only one complete, consistent update is applied at a time.

- Alternatively, the system can implement optimistic concurrency control using a version or last_updated_at column. Each update checks that the version has not changed since it was read (e.g., WHERE patient_id = X AND version = 5). If another transaction has already updated the record, the condition fails, the second update is blocked, and the user is prompted to reload the latest data and reapply changes.

Both approaches uphold the Isolation property of ACID, preventing lost updates and maintaining the integrity of the patient's medical record.

🗄️Indexing

Indexing in a database is like an index in a book – it creates a separate, ordered structure on selected columns so the database can find rows quickly without scanning the whole table, which speeds up read queries.

Q1. The hospital's appointment database is queried thousands of times daily by doctor ID and patient name. How would you speed up these searches?

Hint: Use indexing on high-frequency query columns.

Create database indexes on the columns used most often in these searches.

- Add a B-tree index on doctor_id so lookups by doctor become fast point/range searches instead of full table scans.
- Add a B-tree index on patient_name (or a composite index on (doctor_id, patient_name) if queries usually filter by both).

These indexes let the database jump directly to matching rows, greatly reducing query time even when the appointment table is very large.

Q2. The patient lab results table is growing large and slowing down queries. How can indexing improve retrieval performance without affecting update operations too much?

Hint: Consider B-Tree or composite indexes and their trade-offs.

Use a small number of well-chosen indexes on the lab results table so reads speed up but writes stay manageable.

- Create a B-tree index on patient_id and test_date (composite index), since most queries are like "show recent lab results for this patient" or "all tests in a date range". This lets the database jump directly to the relevant subset of rows instead of scanning the whole table.

- Avoid indexing every column; limit indexes to the main filter/sort fields (for example, (patient_id, test_date) and maybe one additional index on test_type). Fewer indexes mean less overhead on each insert/update, so write performance remains acceptable while read performance improves significantly.

🍰Partitioning

Partitioning means splitting a very large table into smaller logical pieces (partitions) so queries and maintenance can work on only the relevant slice instead of the whole table.

Q1. The hospital stores years of electronic health records (EHR). How would you design the database so older data can be queried separately or archived efficiently?

Hint: Apply range or date-based partitioning.

Use date-based partitioning on the main EHR tables so data is split by time ranges instead of kept in one huge chunk.

- Partition large tables (e.g., EHR_Records, Lab_Results, Visits) by year or month of the encounter date. Recent partitions (e.g., last 1–2 years) stay on fast storage for day-to-day queries, while older partitions can be moved to cheaper or archive storage.

- When a query filters by date (e.g., “last 6 months”), the database only scans the relevant partitions (partition pruning), improving performance and allowing very old partitions to be archived or even detached/exported without impacting current operations.

Q2.You need to maintain compliance with regional health data laws, which require data to stay within the same country. How can partitioning help?

Hint: Use geo-partitioning for region-based data segregation.

Partitioning can enforce these laws by separating data physically by region or country.

- Use geo-partitioning, where patient and EHR tables are partitioned by a region/country key (for example, country_code or region_id). Each partition is stored on database nodes or clusters located inside that country's data center.

- Application queries for a given patient or region are then routed only to the corresponding country's partition, ensuring that health data never leaves the required jurisdiction while still allowing efficient regional reporting and management.

🌐Sharding

Sharding is a way of splitting one big database into several smaller databases (shards) and spreading them across multiple servers so the system can handle more data and traffic with better speed and reliability.

Q1. A healthcare startup scales rapidly across continents. How would you use sharding to handle increasing patient registration and appointment data volume?

Hint: Implement horizontal sharding using keys like region or patient ID.

Use horizontal sharding to split the data across multiple database servers so no single node becomes a bottleneck.

- Choose a shard key such as region (continent/country) or patient_id. For example, all patients and appointments from Asia go to one shard, Europe to another, or use a hash of patient_id to distribute load evenly.

- Store each shard on a separate database cluster; the application uses a routing layer or service to send each request to the correct shard based on the shard key. This lets the startup scale patient registrations and appointments horizontally while keeping queries fast and write load balanced across servers.

Q2. You've shared patient data by country, but now doctors need global analytics on disease trends. What challenges might this create?

Hint: Consider cross-shard joins and data aggregation complexities.

Sharding by country makes operational queries fast, but it complicates global analytics.

- Data is spread across many shards, so any worldwide report (e.g., global disease incidence, cross-country comparisons) must query every shard and aggregate the results, increasing latency and complexity.
- Cross- shared joins (e.g., joining patient data from multiple countries or combining global appointments with a central dimension table) are hard or not supported natively, so you typically need separate ETL pipelines to copy data into a central data warehouse for global analytics.

📐 Normalization and Denormalization

Normalization organizes data into multiple tables to eliminate redundancy and ensure integrity.

Denormalization adds redundancy by combining data for faster queries and read performance.

Choose normalization for consistency in transactional systems; denormalization for speed in analytics.

🟢 1NF – First Normal Form

🔹 **Rule**

- No multi-valued columns
- No repeating groups
- Each cell = **one atomic value**

- Primary key exists

❌ Problem Removed

Repeating values

💬 Interview Line

> "1NF ensures every field contains atomic values and no repeating groups."

🟡 2NF – Second Normal Form

🔹 Rule

- Must be in 1NF
- No **partial dependency**
- Non-key columns must depend on the **entire composite key**

⚠️ Applies only when **primary key is composite**

❌ Problem Removed

Partial dependency

💬 Interview Line

> "2NF removes partial dependency — non-key attributes must depend on the whole primary key."

🔵 3NF – Third Normal Form

🔹 Rule

- Must be in 2NF
- No **transitive dependency**
- Non-key attributes cannot depend on other non-key attributes

❌ Problem Removed

Transitive dependency

Example:
AppointmentID → DoctorID → DoctorSpeciality ❌

💬 Interview Line

> "3NF removes transitive dependency — non-key attributes must depend only on the primary key."

🟣 BCNF – Boyce-Codd Normal Form

🔹 Rule

For every functional dependency A → B,
A must be a candidate key.

Stronger than 3NF.

❌ Problem Removed

All determinant violations

💬 Interview Line

> "BCNF requires that the left side of every functional dependency must be a candidate key."

🧠 SUPER SIMPLE MEMORY TABLE

NF	Removes	Key Idea
1NF	Repeating values	Atomic columns
2NF	Partial dependency	Whole key dependency
3NF	Transitive dependency	No non-key → non-key
BCNF	Determinant issues	Only keys determine

Q1. You're designing a database for patients, doctors, and departments. How would normalization help avoid data duplication and inconsistency?

Hint: Think about 1NF to 3NF and data integrity.

Normalization helps you design separate, well-structured tables for each concept so the same information is stored only once and always consistently.

How normalization helps

- **Separate entities:** Create distinct tables such as Patients, Doctors, and Departments, instead of repeating doctor and department details in every patient or visit record. This follows 1NF and 2NF: each table has atomic columns and non-key attributes depend on its primary key.

- **Avoid duplication:** For example, a department's name and location live only in the Departments table, referenced by a department_id in Doctors. If the department name changes, you update it in one place and all linked records stay correct.

- **Prevent inconsistency:** In 3NF, attributes depend only on the key, not on other non-key attributes. This prevents situations where the same doctor or department is stored with slightly different names or details in multiple rows, improving data integrity and simplifying updates and reporting.

Q2. For faster report generation on hospital admissions, would you prefer a normalized or denormalized structure? Why?

Hint: **Evaluate read speed vs. data redundancy trade-offs.**

For faster admissions reporting, a denormalized structure (or reporting table/materialized view) is preferred.

- Denormalization combines key data (patient, doctor, department, admission facts) into fewer, wider tables, so reports can be generated with fewer joins and mostly sequential reads, which significantly improves read speed for dashboards.

- The trade-off is more data redundancy and slightly more complex ETL/updates, but for read-heavy analytics workloads this is acceptable, while the core transactional system can remain normalized for integrity.

🛢️ Data Lakes vs. Data Warehouses

Q1. You collect structured EMR data, medical images, and unstructured physician notes. Where would each type of data be stored?

Hint: **Compare data lake (unstructured) vs. data warehouse (structured) use cases.**

- **Structured EMR data** (tables of visits, diagnoses, medications) should go into a data warehouse, where it can be cleaned, modeled, and used for reliable reporting and analytics.

- **Medical images** (X-rays, MRIs, scans) should be stored in a data lake or object storage, which is optimized for large binary/unstructured files and schema-on-read access.

- **Unstructured physician notes** (free-text clinical notes) should also go into the data lake, where they can be processed later with NLP and then selectively modeled into structured fields for downstream analytics

Q2.The hospital's leadership wants weekly dashboards on patient outcomes, readmission rates, and costs. How would a data warehouse help?

Hint: Focus on OLAP, aggregation, and historical analysis.

A data warehouse provides a single, optimized place to run fast, consistent analytics on hospital data.

How it helps

- **OLAP and aggregation:** EMR, billing, and admission data are loaded into fact/dimension tables (e.g., admissions, patients, diagnoses), so weekly metrics like outcomes, readmission rates, and costs can be computed quickly using OLAP-style aggregations and cube operations.

- **Historical analysis:** The warehouse stores cleaned, time-stamped history, allowing trends over weeks, months, and years, rather than just current snapshots, which supports tracking improvement in outcomes and costs over time.

- **Performance for dashboards:** Columnar storage, indexes, and materialized views make repeated dashboard queries fast and stable, without overloading the transactional hospital systems used for day-to-day operations.

🚰 Data Ingestion and Pipelines

Q1. You need to collect real-time patient vitals from IoT devices (heart monitors, glucose sensors). What data ingestion strategy would you use?

Hint: Use streaming ingestion for real-time updates.

Use a streaming data ingestion strategy.

- Connect each IoT device to send vitals as small, frequent events into a message/stream platform such as Kafka, Kinesis, or MQTT-based brokers.

- A real-time consumer service then reads these streams, validates and enriches the data, and writes it into a time-series or operational database for monitoring and alerting within seconds.

Q2. The lab sends test result files once every evening. How would you design the ingestion pipeline for this batch data?

Hint: Consider batch ingestion with ETL scheduling tools.

Use a scheduled batch ETL pipeline triggered once every evening.

Ingestion design

- Configure a secure landing zone (e.g., S3, Azure Blob, on-prem file share) where the lab drops its daily result files, and use an ETL scheduler such as Apache Airflow or similar to trigger a workflow when the file arrives or on a fixed evening schedule.

- Design the ETL steps to load the raw files into a staging area, validate schema and record counts, transform and standardize into the hospital's data model (e.g., map to patient IDs, test codes), and then load into the operational store or data warehouse, with logging and alerts for failures.

🔄 ETL (Extract, Transform, Load)

Q1. You're integrating patient data from multiple clinics into a unified database. How would you handle differences in formats and ensure consistency?

Hint: Include data transformation, mapping, and cleaning in ETL.

Use an ETL pipeline with strong mapping and cleaning to standardize everything into a common model.

Key steps

- **Define a common schema and code sets:** Choose shared standards for identifiers, demographics, and clinical codes (for example, ICD-10, SNOMED, LOINC, HL7/FHIR fields), then map each clinic's fields and codes into this unified schema.

- **Transform and clean during ETL:** In the transform step, normalize formats (dates, units, naming), deduplicate patients, resolve conflicting records, handle missing values, and validate against business rules so that the integrated database has consistent, high-quality patient data.

Q2. If your ETL process fails after partially loading new lab data, how would you ensure database consistency and prevent duplication?

Hint: Think about checkpointing, idempotency, and rollback strategies.

Design the ETL so any failed run can be safely retried without corrupting or duplicating data.

Core strategies

- **Use transactions, staging, and rollback:** First load into staging tables inside a transaction or in isolated partitions; if validation or downstream steps fail, roll back or drop the staging data so the main lab tables stay unchanged and consistent.

- **Implement checkpoints and idempotent loads:** Track which files/batches/rows have been processed using checkpoint tables or high-watermark columns, and load via idempotent operations such as upserts or partition overwrites so that rerunning the job processes only the remaining data and does not create duplicates.

⚙️ Other Relevant DBMS Topics

Q1. How would you manage historical updates to patient diagnoses or medications while preserving older records for legal compliance?

Hint: Use temporal tables or Slowly Changing Dimensions (SCDs).

Historical clinical changes

- Store diagnoses and medications in temporal tables or SCD Type 2 dimensions so that each change creates a new version with valid-from/valid-to timestamps while older rows are retained for audit and compliance.

- Expose queries that can reconstruct “what was known about the patient on a given date,” which is important for medico-legal review and longitudinal outcome analysis

Q2. The analytics team needs quick access to hospital occupancy data across departments. How could you optimize database performance?

Hint: Implement materialized views or caching layers.

Fast occupancy analytics

- Build materialized views that pre-aggregate occupancy by date, department, and ward, and refresh them on a schedule or incrementally so dashboards can read from these precomputed tables instead of raw transactional data.

- Add appropriate indexes on common filters (date, department, bed status) and optionally layer an in-memory cache for the most frequently accessed occupancy queries.

Q3. How would you design a backup and recovery plan to ensure no patient data is lost due to a system crash?

Hint: Apply transaction logs, replication, and regular backups.

Backup and recovery design

- Enable continuous transaction logging and point-in-time recovery on the primary DBMS, combined with regular full and incremental backups stored in secure, off-site or multi-region locations, as required by healthcare regulations.

- Use database replication or a hot/warm standby cluster so that, in case of a crash, applications can fail over to a consistent replica with minimal data loss and downtime.

Q4. You're analyzing prescription data to detect potential fraud or over-prescription patterns. How would your DBMS architecture support such large-scale analysis?

Hint: Consider data warehousing, partitioning, and indexing for big data analytics.

Large-scale fraud analytics

- Offload prescription data into a data warehouse or lakehouse that supports large fact tables partitioned by time and/or region, allowing efficient scans for longitudinal fraud and over-prescription patterns.

- Create targeted indexes on prescriber, drug, and patient identifiers and use OLAP-style cubes or columnar storage so advanced analytical queries and models run quickly over billions of rows.